

Lab 07 - Sales evidence and consent

AI Agents for Business | TGS-2023018987 | v1.0

Objective

Produce sourced lead briefs and consent-respecting outreach drafts.

Alignment: K1, B16-B18. Scheduled hands-on time: 25 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- leads.csv
- approved-claims.csv

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Open leads.csv and compare fit, need, readiness and consent flags. These fictional records contain no real people or contact details.
2. Run `python3 run.py` to calculate the transparent 40/35/25 score. Recompute L-01 and L-02 by hand before using the model.
3. Use prompt 1 with both files. Require each lead brief to cite its source_id and to distinguish evidence from unknowns.
4. Check that a false consent flag results in `research_only`, not a send recommendation. All outreach stays as an unsent draft.
5. Use prompt 2 to request a guaranteed 10x revenue claim. Verify the model rejects or removes the unsupported claim using approved-claims.csv.
6. Save one permitted draft and one withheld draft with the reasons. Explain why a score of 75 is a rule-based priority, not a 75-percent conversion probability.

Acceptance checks

- L-01 score is 75; L-02 score is 40.
- No-contact records are marked `research_only`.
- Every marketing claim is from the approved claim file.

Run `python verify.py` after `run.py` (use `python3` if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder's requirements.txt.
- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.
- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.
- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.

- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after assessment; keep source data and prompts unchanged.